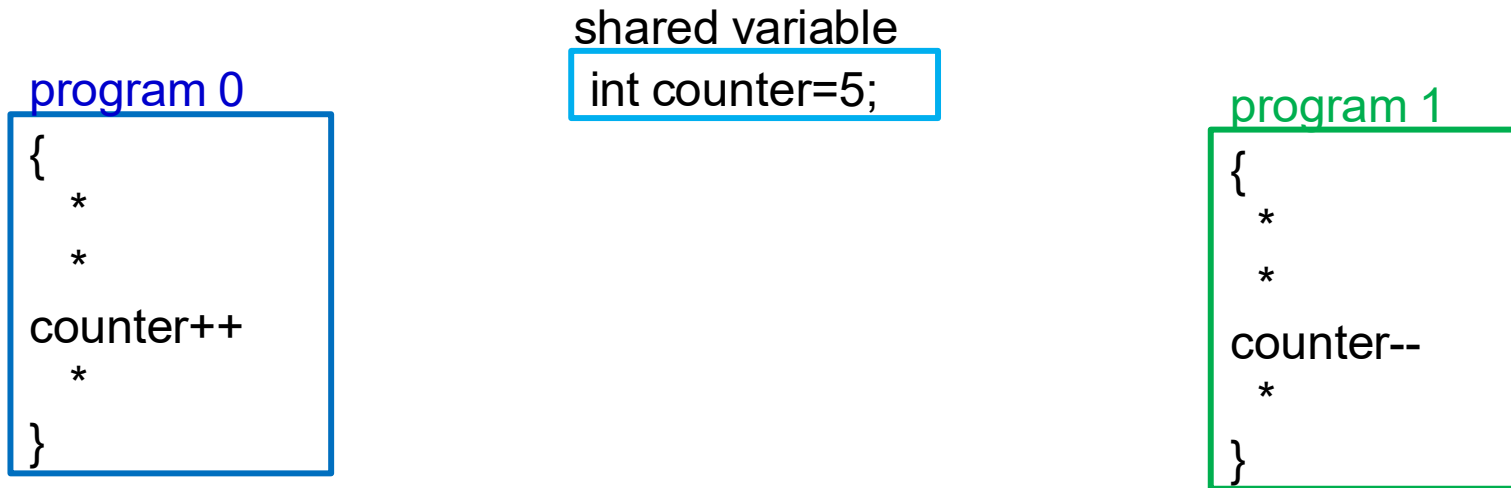


Process Synchronization

Motivating Scenario



- Single core
 - Program 1 and program 2 are executing at the same time but sharing a single core



→ CPU usage wrt time

Source: http://www.cse.iitm.ac.in/~chester/courses/15o_os/syllabus.html

Motivating Scenario

program 0

```
{  
  *  
  *  
  counter++  
  *  
}
```

Shared variable

```
int counter=5;
```

program 1

```
{  
  *  
  *  
  counter--  
  *  
}
```

- What is the value of counter?
 - expected to be 5
 - but could also be 4 and 6

Motivating Scenario

program 0

```
{
  *
  *
  counter++
  *
}
```

Shared variable

```
int counter=5;
```

program 1

```
{
  *
  *
  counter--
  *
}
```

context
switch

$R1 \sqcap \text{counter}$ $R1 \sqcap R1 + 1$ $\text{counter} \sqcap R1$
$R2 \Downarrow \text{counter}$ $R2 \Downarrow R2 - 1$ $\text{counter} \Downarrow R2$

counter = 5

$R1 \sqcap \text{counter}$ $R2 \Downarrow \text{counter}$ $R2 \Downarrow R2 - 1$ $\text{counter} \Downarrow R2$
$R1 \sqcap R1 + 1$ $\text{counter} \sqcap R1$

counter = 6

$R1 \Downarrow \text{counter}$ $R2 \sqcap \text{counter}$ $R2 \sqcap R2 + 1$ $\text{counter} \sqcap R2$
$R1 \Downarrow R1 - 1$ $\text{counter} \Downarrow R1$

counter = 4

Race Conditions

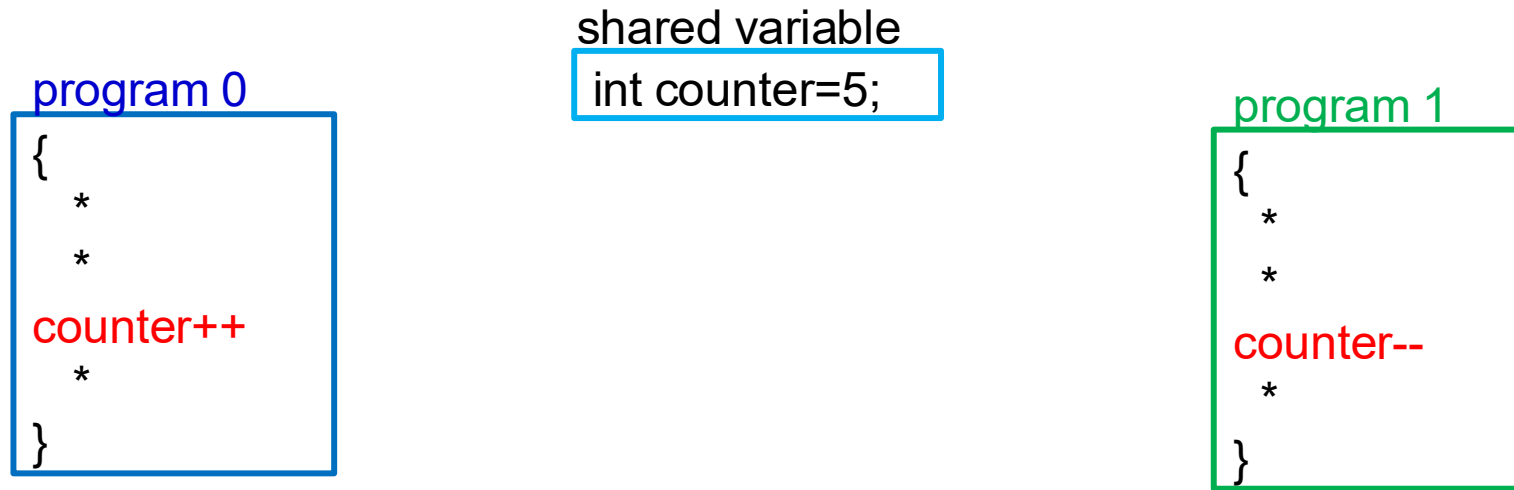
- Race conditions
 - A situation where several processes access and manipulate the same data (**critical section**)
 - The outcome depends on the order in which the access take place
 - Prevent race conditions by synchronization
 - Ensure only one process at a time manipulates the critical data



critical section

*No more than one
process should execute in
critical section at a time*

Race Conditions in Multicore



- Multi core
 - Program 1 and program 2 are executing at the same time on different cores



Background

- Processes can execute **concurrently**
- May be **interrupted** at any time, **partially completing** execution
- Concurrent access to shared data may result in data **inconsistency**
- Maintaining data consistency requires mechanisms to ensure the **orderly execution of cooperating processes**

Critical Section Problem

- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process in critical section, no other may be in its critical section
- **Critical section problem** is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

Critical Section

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

Race Conditions

- **Race:** A situation where several processes access and manipulate the same data (critical section)
- The outcome depends on the order in which the access take place
- Prevent race conditions by synchronisation
 - Ensure only one process at a time

manipulates the critical data



critical section

No more than one process should execute in critical section at a time

Solution To Critical Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then **no other processes** can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next **cannot be postponed indefinitely**
3. **Bounded Waiting** - **A bound must exist on** the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning **relative speed** of the n processes

First Version of Dekker's Algorithm

```
/* PROCESS 0 */  
.  
.  
while (turn != 0)  
    /* do nothing */ ;  
    /* critical section*/;  
    turn = 1;  
.
```

```
/* PROCESS 1 */  
.  
.  
while (turn != 1)  
    /* do nothing */;  
    /* critical section*/;  
    turn = 0;  
.
```

The **turn** is used to indicate if a process is ready to enter the critical section.

(a) First attempt

Second Version of Dekker's Alg

```
/* PROCESS 0 */  
.  
.  
while (flag[1])  
    /* do nothing */;  
flag[0] = true;  
/*critical section*/;  
flag[0] = false;  
.
```

```
/* PROCESS 1 */  
.  
.  
while (flag[0])  
    /* do nothing */;  
flag[1] = true;  
/* critical section*/;  
flag[1] = false;  
.
```

(b) Second attempt

Third Version of Dekker's Alg

<code>/* PROCESS 0 */</code>	<code>/* PROCESS 1 */</code>
<code>.</code>	<code>.</code>
<code>.</code>	<code>.</code>
<code>flag[0] = true;</code>	<code>flag[1] = true;</code>
<code>while (flag[1])</code>	<code>while (flag[0])</code>
<code> /* do nothing */;</code>	<code> /* do nothing */;</code>
<code>/* critical section*/;</code>	<code>/* critical section*/;</code>
<code>flag[0] = false;</code>	<code>flag[1] = false;</code>
<code>.</code>	<code>.</code>

(c) Third attempt

Fourth Version of Dekker's Alg

```
/* PROCESS 0 */
.
.
flag[0] = true;
while (flag[1]) {
    flag[0] = false;
    /*delay */;
    flag[0] = true;
}
/*critical section*/;
flag[0] = false;
.

/* PROCESS 1 */
.
.
flag[1] = true;
while (flag[0]) {
    flag[1] = false;
    /*delay */;
    flag[1] = true;
}
/* critical section*/;
flag[1] = false;
```

(d) Fourth attempt

Dekker's algorithm of ME Primitive

```
void P0 () {  
    while (true) {  
        flag [0] = true;  
        while (flag [1]) {  
            if (turn == 1) {  
                flag [0] = false;  
                while (turn == 1)  
                    /* do nothing */;  
                flag [0] =  
                    true;}}  
        /* critical section */;  
        turn = 1;  
        flag [0] = false;  
        /* remainder */;}}
```


Dekker's algorithm of ME Primitive

P0

```
flag [0] = true;
while (flag [1]) {
    if (turn == 1) {
        flag [0] = false;
        while (turn == 1);
        flag [0] = true;
    }
}
/* critical section */;
turn = 1;
flag [0] = false;
/* remainder */;
```

P1

```
flag [1] = true;
while (flag [0]) {
    if (turn == 0) {
        flag [1] = false;
        while (turn == 0);
        flag [1] = true;
    }
}
/* critical section */;
turn = 0;
flag [1] = false;
/* remainder */;
```

Peterson's Solution (Two Process)

- Two process solution
- The two processes share two variables:

int turn;

Boolean flag[2]

- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section.

flag[i] = *true* implies that process **P_i** is ready!

Peterson's Solution (Two Process)...

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = false;  
    remainder section  
} while (true);
```

Peterson's Solution (Two Process)...

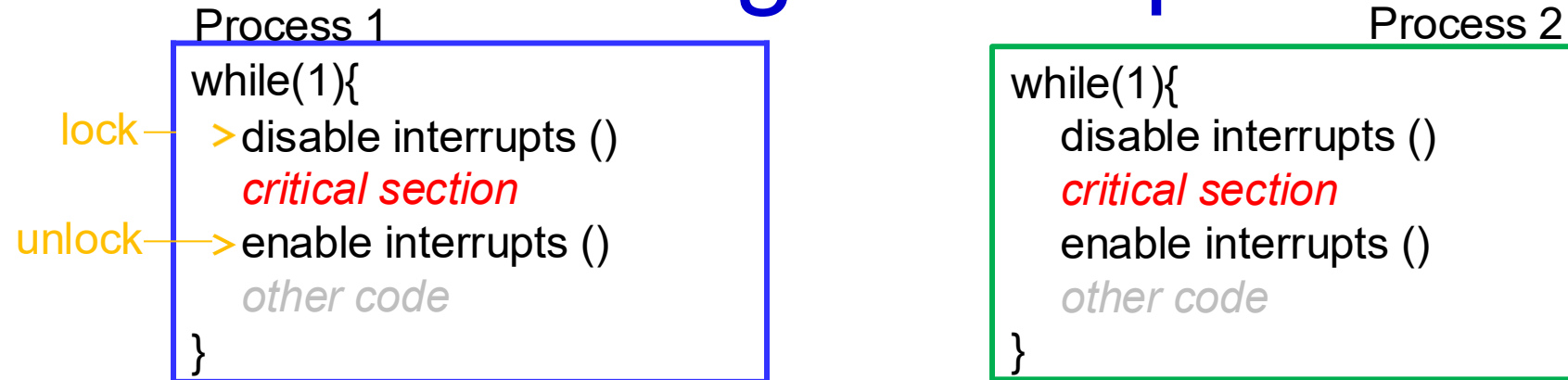
```
do {  
    flag[0] = true;  
    turn = 1;  
    while (flag[1] &&  
turn==1); //nothing  
        critical section  
        flag[0] = false;  
        remainder section  
    } while (true);
```

```
do {  
        flag[1] = true;  
        turn = 0;  
        while (flag[0] &&  
turn ==0); //nothing  
            critical  
            section  
            flag[1] =  
false;  
            remainder  
            section  
        } while (true);
```

Synchronization Hardware

- Many systems provide hardware support for implementing the critical section code.
- All solutions below based on idea of **locking**
 - Protecting critical regions via locks
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - **Atomic** = non-interruptible
 - Either test memory word and set value
 - Or swap contents of two memory words

Using Interrupts



- Simple
 - When interrupts are disabled, context switches won't happen
- Requires privileges
 - User processes generally cannot disable interrupts
- Not suited for multicore systems

Analyze this

- Does this scheme provide mutual exclusion?

Process 1

```
while(1){
    while(lock != 0);
    lock = 1; // lock
    critical section
    lock = 0; // unlock
    other code
}
```

Process 2

```
while(1){
    while(lock != 0);
    lock = 1; // lock
    critical section
    lock = 0; // unlock
    other code
}
```

No

lock = 0	
P1: while(lock != 0);	
P2: while(lock != 0);	
P2: lock = 1;	
P1: lock = 1;	
.... Both processes in critical section	

context switch

If only...

- We could make this operation atomic

Process 1

```
while(1){  
    while(lock != 0);  
    lock= 1; // lock  
    critical section  
    lock = 0; // unlock  
    other code  
}
```

Make atomic

Hardware to the rescue....

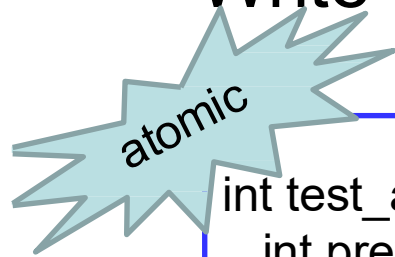
- Definition:

```
boolean test_and_set (boolean *target) {  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;}
```

- Executed atomically
- Returns the original value of passed parameter
- Set the new value of passed parameter to “TRUE”.

Hardware Support (Test & Set Instruction)

- Write to a memory location, return its old value



```
int test_and_set(int *L){  
    int prev = *L;  
    *L = 1;  
    return prev;  
}
```

```
while(1){  
    while(test_and_set(&lock) == 1);  
    critical section  
    lock = 0; // unlock  
    other code  
}
```

equivalent software representation
(the entire function is executed atomically)

Usage for locking

Why does this work? If two CPUs execute test_and_set at the same time, the hardware ensures that one test_and_set does both its steps before the other one starts.

So the first invocation of test_and_set will read a 0 and set lock to 1 and return. The second test_and_set invocation will then see lock as 1, and will loop continuously until lock becomes 0

Hardware Support ... (compare_and_swap)

- Definition:

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

- Executed atomically
- Returns the original value of passed parameter “value”
- Set the variable “value” to “new_value” only if “value” == “expected”. That is, the swap takes place only under this condition.

Solution using compare_and_swap

- Shared integer “lock” initialized to 0;
- Solution:

```
do {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
    /* remainder section */  
  
} while (true);
```

Hardware Solution...

- Mutual exclusion is guaranteed/preserved.
- Bounded waiting is not guaranteed.
- Advantages
 - Applicable to any number of processes on either a single processor or multiple processors sharing main memory.
 - It is simple and therefore easy to verify.
 - It can be used to support multiple critical sections.

Bounded-waiting Mutual Exclusion with test_and_set

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key) //The variable waiting[i] can become false only
if another process leaves its critical section;
        key = test_and_set(&lock); //only one process could get key =False
(which first time sets lock=True)
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j]) //Check if any other process is
waiting(in order)
        j = (j + 1) % n;
    if (j == i) //If NO other process is waiting, any process can do so
including current process.
        lock = false;
    else
        waiting[j] = false; //if so, set its waiting=False
    /* remainder section */
} while (true);
```

boolean waiting[n];
boolean lock;
[Initialized to false]

```
while(1){
    while(test_and_set(&lock) == 1);
    /* critical section */
    lock = 0; //unlock
    other code}
```

Mutex Locks

- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem
- Simplest is **mutex lock**
- Protect a critical section by first **acquire()** a lock then **release()** the lock
 - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic
 - Usually implemented via hardware atomic instructions
- But this solution requires **busy waiting**
 - This lock therefore called a **spinlock**

acquire() and release()

- `acquire()` {
 while (!available)
 ; /* busy wait */
 available = false;
}
- `release()` {
 available = true; }
- Solution to CS Problem Using Mutex Locks
do {
 acquire lock
 critical section
 release lock
 remainder section
} while (true);

Atomic Variables

- Typically, instructions such as compare-and-swap are used as building blocks for other synchronization tools.
- One tool is an **atomic variable** that provides *atomic* (uninterruptible) updates on basic data types such as integers and booleans.
- For example:
 - Let **sequence** be an atomic variable
 - Let **increment()** be operation on the atomic variable **sequence**
 - The Command:
increment(&sequence) ;
ensures **sequence** is incremented without interruption:

Atomic Variables...

- The `increment()` function can be implemented as follows:

```
void increment(atomic_int *v)
{
    int temp;
    do {
        temp = *v;
    }
    while (temp != (compare_and_swap(v, temp, temp+1)) );
}
```

Semaphore

- **Semaphore** is a synchronization tool that provides more sophisticated ways for process to synchronize their activities.
- Semaphore **S** – integer variable
- Can only be accessed via two indivisible (**atomic**) operations
 - **wait()** and **signal()**
 - Originally called **P()**; (from the Dutch *proberen*, “to test”) and **V()** (from the Dutch *verhogen*, “to increment”)
- Definition of the **wait()** operation

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;} 
```
- Definition of the **signal()** operation

```
signal(S) {  
    S++;} 
```

Semaphore...

- **Counting** semaphore – integer value can range over an unrestricted domain
- **Binary** semaphore – integer value can range only between 0 and 1; can be simpler to implement
 - Also known as **mutex locks**
- Mutual-exclusion implementation with **Binary** semaphore
- Semaphore mutex; // initialized to 1

```
do {  
    wait (mutex);  
    // Critical Section  
    signal (mutex);  
    // remainder section  
} while (TRUE);
```

Semaphore...

- Case 1: To control access to a given resource
- Semaphore is initialized to the number of resources
- Semaphore Count = 0; all resources are utilized
- Case 2 : process synchronization
- How to ensure S2 is executed only after S1 completes?

Process1	Process2	Process1	Process2
S1;	S2;	S1;	wait(synch)
		signal(synch)	S2;

Disadvantage: Busy waiting thus wastes CPU cycles

Semaphore...

- While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code.
- This type of semaphore is also called **spinlock** because the process "spins" while waiting for the lock.
- Busy waiting wastes CPU cycles that some other process might be able to use productively.
- (Spinlocks do have an advantage in that no context switch is required when a process must wait on a lock, and a context switch may take considerable time)
- To overcome the need for busy waiting, we can modify the definition of the wait() and signal() semaphore operations.
- rather than engaging in busy waiting, the process can *block* itself. The block operation places a process into a waiting queue associated with the semaphore

Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue

- Each entry in a waiting queue has two data items:

- value (of type integer)
- pointer to next record in the list

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

- Two operations:

- **block** – place the process invoking the operation on the appropriate waiting queue
- **wakeup** – remove one of processes in the waiting queue and place it in the ready queue

Semaphore Implementation with no Busy waiting ...

- Synchronization tool that does not require busy waiting
- Implementation of wait:

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

Entry

Semaphore value –ve indicates the processes waiting to acquire that semaphore

- Implementation of signal:

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

Exit

Put the processes from the blocked list to ready queue

Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let **S** and **Q** be two semaphores initialized to 1

P_0
wait (S);
wait (Q);
.
.
.
signal (S);
signal (Q);

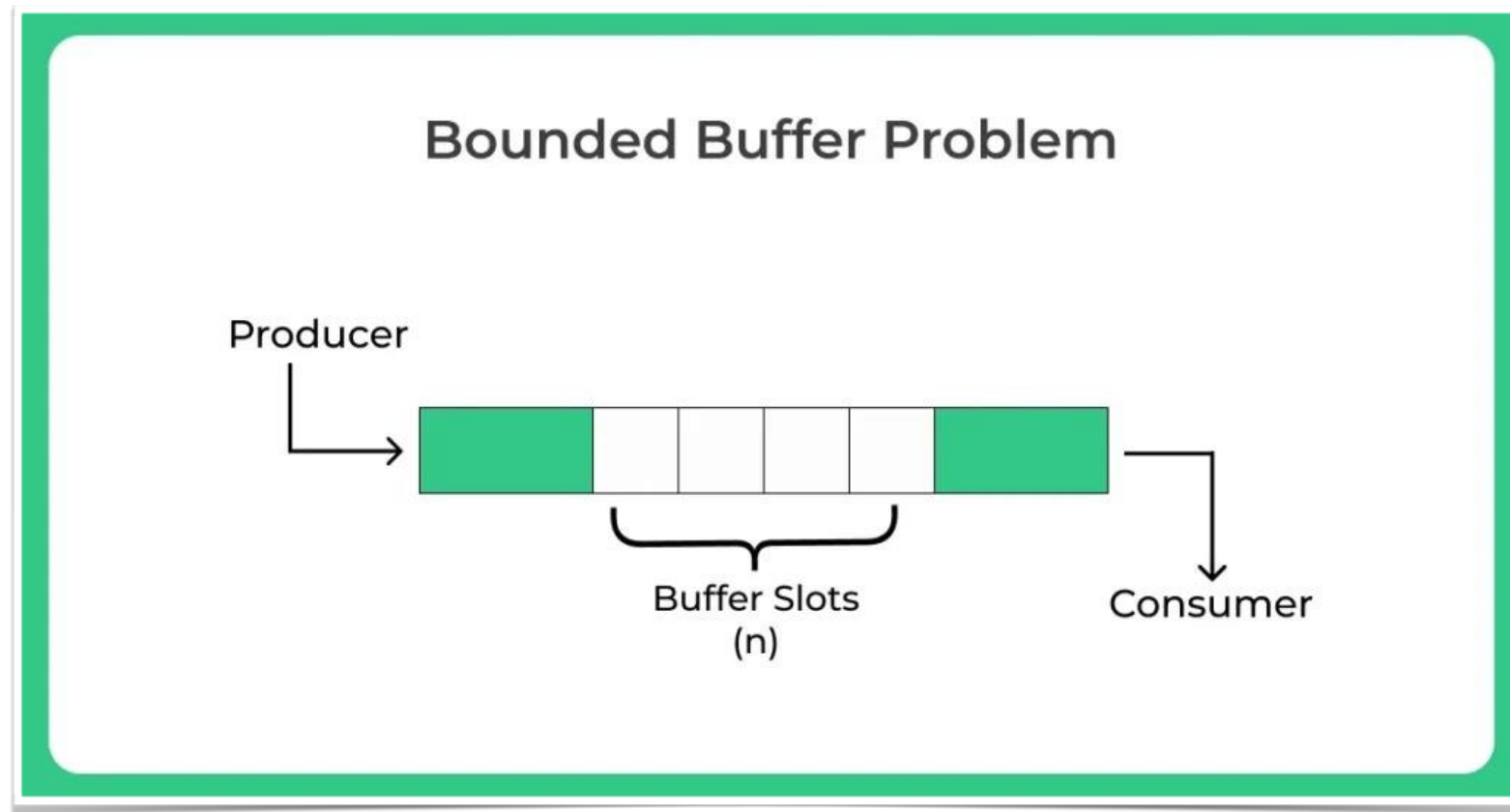
P_1
wait (Q);
wait (S);
.
.
.
signal (Q);
signal (S);

- **Starvation** – indefinite blocking
 - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process (occurs when a high priority task is indirectly preempted by a low priority task.)
 - Solved via **priority-inheritance protocol**

Classical Problems of Synchronization

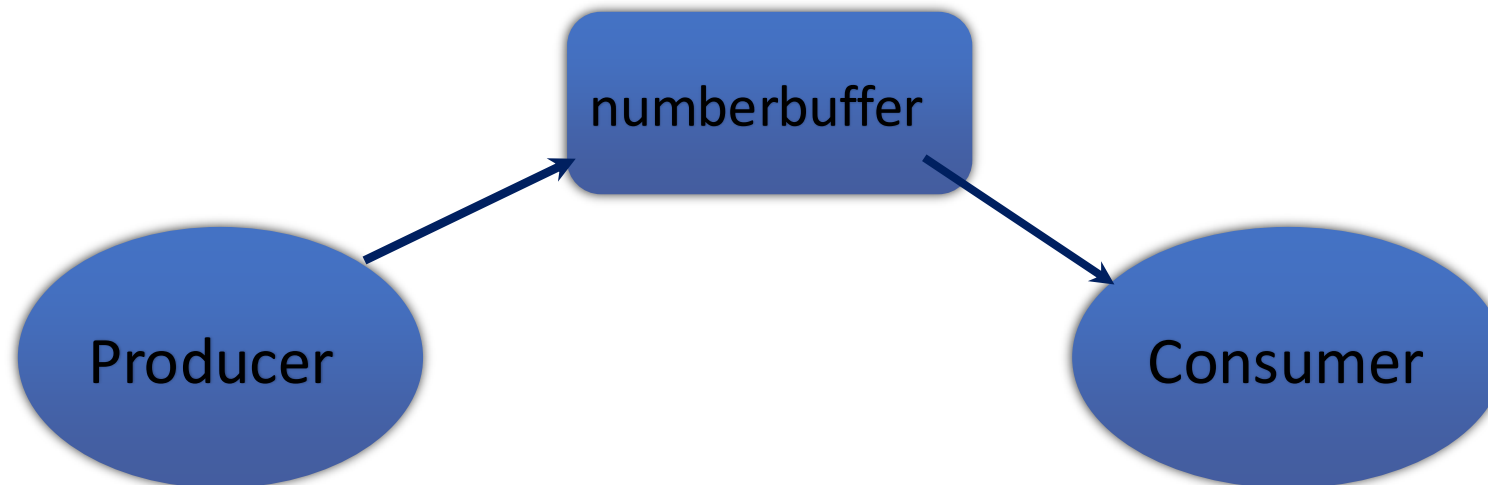
- Classical problems used to test newly-proposed synchronization schemes
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem

Bounded-Buffer Problem



Bounded-Buffer Problem

- N buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value N



Bounded-Buffer Problem...

- The structure of the producer process

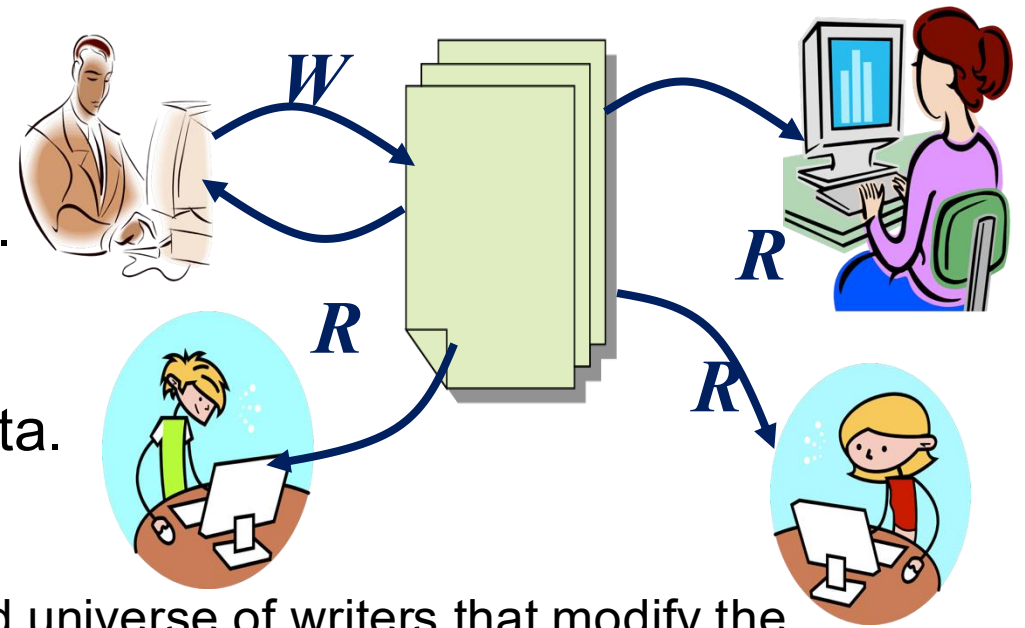
```
do {  
    // produce an item in nextp  
    wait (empty);  
    wait (mutex);  
    // add nextp to buffer  
    signal (mutex);  
    signal (full);  
} while (TRUE);
```

- The structure of the consumer process

```
do {  
    wait (full);  
    wait (mutex);  
    // remove an item from buffer to  
    nextc  
    signal (mutex);  
    signal (empty);  
    // consume the item in nextc  
} while (TRUE);
```

Readers-Writers Problem

- Depending on the usage of the resource, processes are classified into either readers or writers.
- A reader never modifies the shared data.
- Writer may both read and write the shared data.
- Readers may use the shared data concurrently.
- Writers must be granted exclusive access to data.
- Problem can be stated as below
 - Given universe of readers that read a shared data and universe of writers that modify the same shared data
 - Plan a synchronization protocol among the readers and writers that ensures data consistency while maintaining high degree of concurrency.



Readers-Writers Problem...

- A data set is shared among a number of concurrent processes
 - Readers – only read the data set; they do **not** perform any updates
 - Writers – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities
- Shared Data
 - Semaphore **mutex** initialized to 1
 - Semaphore **rw_mutex** initialized to 1
 - Integer **read_count** initialized to 0

Readers-Writers Problem...

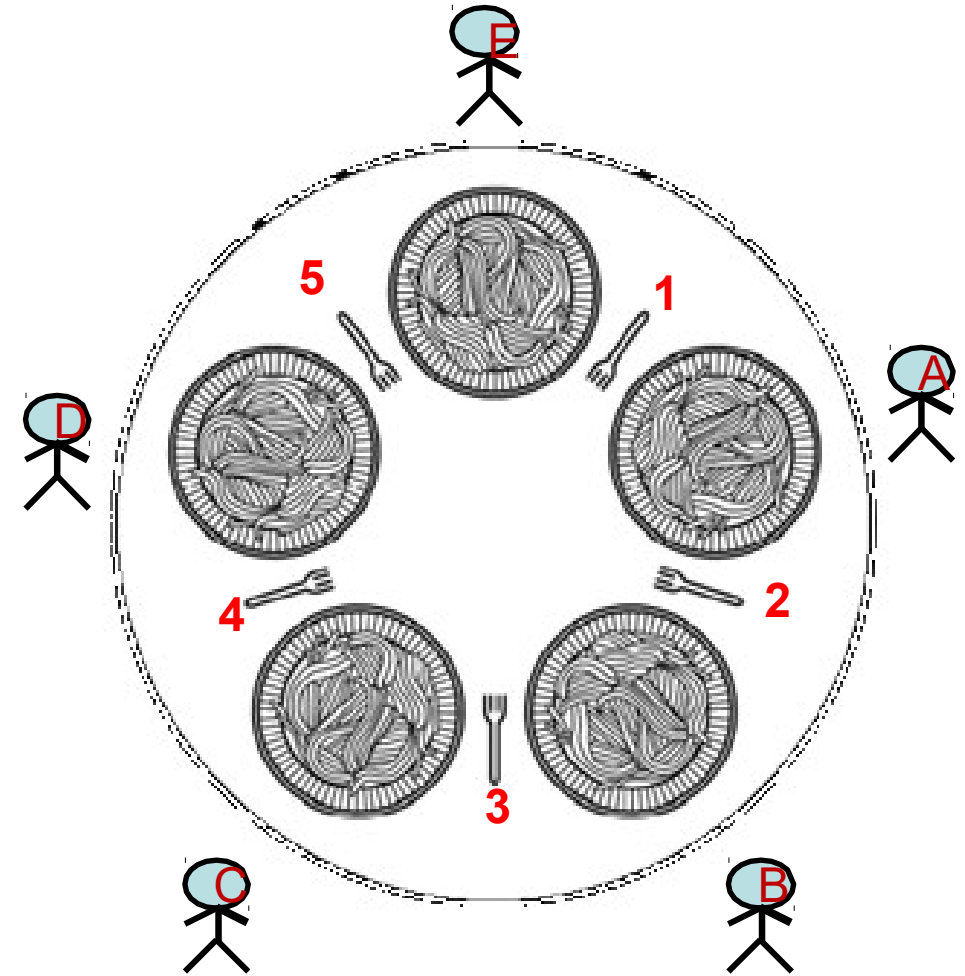
- The structure of a writer process

```
do {
    wait(rw_mutex);
    ...
    /* writing is
performed */
    ...
    signal(rw_mutex);
} while (true);
```
- The structure of a reader process

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);
    ...
    /* reading is
performed */
    ...
    wait(mutex);
    read count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```


Dining Philosophers Problem

- Philosophers either think or eat
- Each need to use 2 forks for eating
- We have only 5 forks
- Problem Statement : Develop an algorithm where no philosopher starves.
- A classical synchronization problem
- Illustrates the difficulty of allocating resources among process without deadlock and starvation

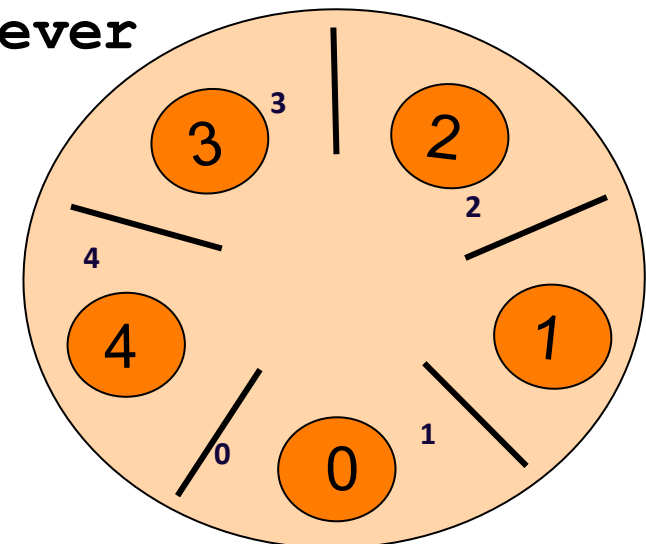


Dining Philosophers Problem...

- Each philosopher is a process
- One semaphore per fork:
 - fork: array[0..4] of semaphores
 - Initialization: fork[i].count:=1 for i:=0..4
- A first attempt:
- Deadlock if each philosopher start by picking his left fork!

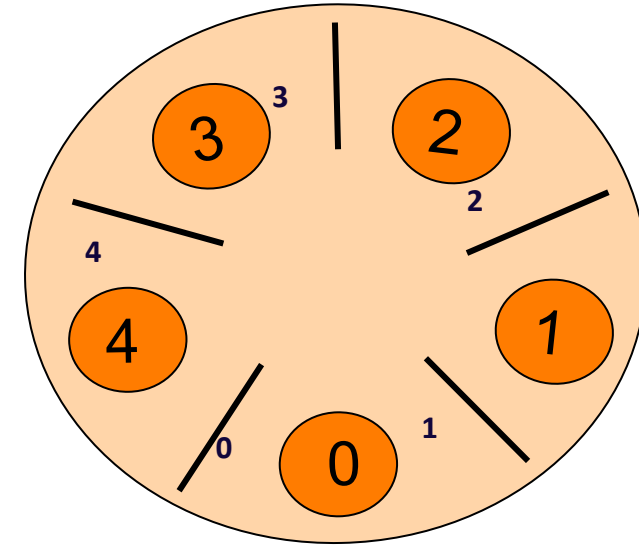
Process P_i :

```
repeat
  think;
  wait(fork[i]);
  wait(fork[i+1 mod 5]);
  eat;
  signal(fork[i+1 mod 5]);
  signal(fork[i]);
forever
```



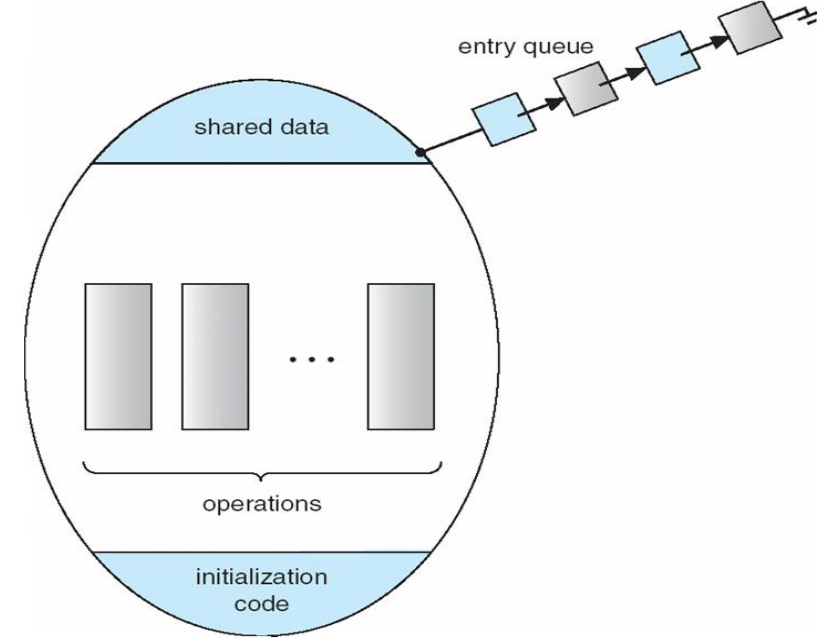
Dining Philosophers Problem...

- Possible solutions
 - Allow at most four philosophers to be sitting simultaneously at the table.
 - Allow a philosopher to pick up her chopsticks only if both chopsticks are available.
 - An odd philosopher picks up first her left chopstick and then her right chopstick. Where as an even philosopher picks up her right chopstick and then her left chopstick.



Monitors

- Are high-level language constructs that provide equivalent functionality to that of semaphores but are easier to control
- Found in many concurrent programming languages
 - Concurrent Pascal, Java etc.
- Monitor is a software module containing:
 - one or more procedures
 - an initialization sequence
 - local data variables



```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { .... }

    ...

    procedure Pn (...) {.....}

    Initialization code ( ....) { ... }

    ...
}
}
```

Monitors...

- Characteristics:
 - local variables accessible only by monitor's procedures
 - a process enters the monitor by invoking one of its procedures
 - only one process at a time can be active within the monitor .
- The monitor ensures mutual exclusion: no need to program this constraint explicitly
- The monitor locks the shared data on process entry.

Monitors With Cond'n. Variable

- To allow a process to wait within the monitor, a condition variable must be declared, as:

```
condition x, y;
```

- Condition variable can only be used within the operations wait and signal. A queue is associated with each condition variable.

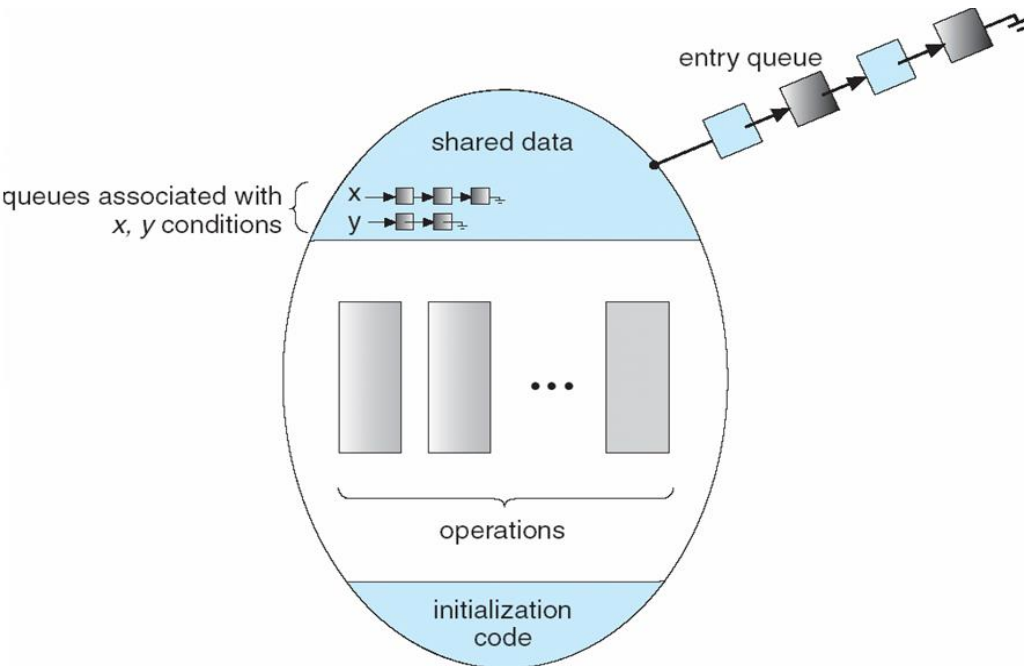
```
x.wait();
```

Means that the process invoking this operation is suspended until another process

invokes

```
x.signal();
```

The x.signal operation resumes exactly one suspended process. (that invoked x.wait) If no process is suspended, then the signal operation has no effect.



Solution to Dining Philosophers

```
monitor DP
```

```
{
    enum { THINKING, HUNGRY, EATING } state[5] ;
    condition self[5];
    void pickup (int i)
    {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait;
    }
    void putdown (int i)
    {
        state[i] = THINKING;
        // test left and right neighbors
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
    void test (int i)
    {
        if ( (state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING))
        {
            state[i] = EATING ;
            self[i].signal();
        }
    }
}
```

```
void init()
```

```
{
```

```
    for (int i = 0; i < 5; i++)
```

```
        state[i] = THINKING;
```

```
}
```

```
}
```

```
DP.pickup(i)
```

```
...
```

```
Eat
```

```
....
```

```
DP.putdown(i)
```

Solution to Dining Philosophers Using Semaphores

```
#define N 5  /* Number of philosophers */
#define RIGHT(i) (((i)+1) %N)
#define LEFT(i) (((i)==N) ? 0 : (i)+1)
typedef enum { THINKING, HUNGRY, EATING } phil_state;
phil_state state[N];
semaphore mutex =1;
semaphore s[N]; /* one per philosopher, all 0 */

void test(int i) {
    if (state[i] == HUNGRY && state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING )
    { state[i] = EATING;
      signal(s[i]);
    }
}

void get_forks(int i) {
    wait(mutex);
    state[i] = HUNGRY;
    test(i);
    signal(mutex);
    wait(s[i]);
}

void put_forks(int i) {
    wait(mutex);
    state[i] = THINKING;
    test(LEFT(i));
    test(RIGHT(i));
    signal(mutex);
}

void philosopher(int process) {
    while(1)
    {
        think();
        get_forks(process);
        eat();
        put_forks(process);
    }
}
```


Synchronization Examples

- Solaris
- Windows
- Linux
- **Pthreads**

Pthreads Synchronisation

- pthreads API is OS-independent
- It provides:
 - mutex locks
 - condition variable
- Non-portable extensions include:
 - read-write locks
 - spinlocks

Pthreads Mutex

```
#include <pthread.h>
pthread_mutex_t mutex;
/* create the mutex lock */
pthread_mutex_init(&mutex, NULL);
```

```
=====
```

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);
/* critical section */
/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

Pthreads Semaphore...

```
#include <semaphore.h>
sem_t sem;
/* Create the semaphore and initialize it to 1 */
sem_init(&sem, is_shared(0/1), 1);
/*Params: A pointer to the semaphore, A flag indicating the level of sharing, The semaphore's
initial value*/
```

=====

```
/* acquire the semaphore */
sem_wait(&sem);
/* critical section */
/* release the semaphore */
sem_post(&sem);
```